

# SRG vs. CareGuardAI

## Architectural Differentiation, Research Positioning, and Justification

### *Self-Refine Guardrail (SRG) for Customer-Support Generative AI*

Purpose. This document establishes a technically defensible comparison between SRG and the selected base paper, CareGuardAI: Context-Aware Multi-Agent Guardrails for Clinical Safety & Hallucination Mitigation in Patient-Facing LLMs. The goal is not to claim that SRG is universally superior. Instead, it identifies the architectural differences, the narrower design guarantees introduced by SRG, and the conditions under which SRG is better aligned with customer-support response verification.

## 1. Base Paper: CareGuardAI

CareGuardAI is an inference-time, multi-agent guardrail framework designed for patient-facing medical question answering. Its stated objective is to jointly control clinical safety risk and hallucination risk. The architecture includes a controller agent, vulnerability screening, safety-constrained generation, separate clinical-safety and hallucination-risk evaluators, a decision layer, and iterative refinement. Responses are released only when both SRA and HRA are at or below 2. The paper reports evaluation on PatientSafeBench, MedSafetyBench, and MedHallu. [cite](#) [turn2view0](#)

Architecturally, CareGuardAI therefore emphasizes context-aware medical triage before generation, risk-scored dual evaluation after generation, and threshold-based release control. The controller can identify missing patient context such as age group, pregnancy status, symptom severity, and urgency, and can generate structured screening questions when required. [cite](#) [turn2view0](#)

## 2. SRG Design in One View

SRG is designed as a response-level verification layer for customer-support chatbots. A candidate response is generated first and then passed through a deterministic Redactor. The redacted representation is supplied to the Review Agent, which evaluates policy compliance, tone, and completeness. The original response is separately supplied to the Error Checker, which evaluates factual correctness and confidentiality. The final application decision is computed deterministically as  $\text{approve} = v1 \text{ AND } v2$ . A denied response enters a bounded Self-Refine loop for at most two attempts; unresolved failures are escalated rather than delivered.

The key design decision is information isolation. The Review Agent does not receive the original sensitive response; the Error Checker does not perform the Review Agent's policy/tone responsibility. This creates explicit boundaries between the two verification concerns.

## 3. Architectural Comparison

| Architectural Aspect   | CareGuardAI                          | SRG                                                               | SRG Differentiation                                   |
|------------------------|--------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------|
| Primary domain         | Patient-facing healthcare            | Customer-support chatbots                                         | Domain-specific adaptation                            |
| Primary risks          | Clinical safety + hallucination      | Policy + factual correctness + confidentiality + response quality | Targets operational support risks                     |
| Pre-generation control | Controller + vulnerability screening | Candidate-response generation followed by verification            | Post-generation response gate                         |
| Evaluator design       | SRA + HRA risk evaluators            | Review Agent + Error Checker                                      | Different responsibilities and information boundaries |
| Evaluator inputs       | Context/risk information             | Review: redacted r'; Error:                                       | Explicit information                                  |

|                         | and generated response                                                  | original r                                                   | isolation                                |
|-------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------|
| Decision mechanism      | Thresholds: $SRA \leq 2$ and $HRA \leq 2$                               | Boolean: approve = v1 AND v2                                 | No numerical safety score                |
| Confidentiality control | Not the central architectural axis                                      | Dedicated deterministic redaction + confidentiality check    | Confidentiality is a first-class concern |
| Policy grounding        | Safety instructions from controller/context                             | Retrieved policy context using sentence-transformers + FAISS | Explicit policy knowledge retrieval      |
| Refinement              | Iterative refinement, up to 3 iterations in the described decision loop | Self-Refine, maximum 2 attempts                              | Strict bounded retry policy              |
| Unresolved failure      | Block                                                                   | Human escalation after bounded retries                       | Operational handoff                      |
| Decision authority      | Risk-score threshold layer                                              | Deterministic application logic                              | Auditable Boolean control                |

Table 1. Architectural comparison between CareGuardAI and SRG.

## 4. The Most Important Architectural Difference: Information Isolation

The strongest architectural distinction is not simply that both systems use two evaluators. CareGuardAI uses two specialized risk assessments—clinical safety and hallucination risk—within a common risk-control framework. SRG instead establishes different information boundaries for two verification gates.

For a candidate response  $r$ , SRG produces a redacted representation  $r'$ . The Review Agent receives  $r'$  plus relevant policy context  $P$  and returns  $v1$ . The Error Checker receives the original  $r$  plus  $P$  and returns  $v2$ . The application layer computes  $\text{approve} = v1 \text{ AND } v2$ . This means the policy/tone evaluator is deliberately prevented from reasoning over raw sensitive content, while the confidentiality evaluator retains access to the original response because confidentiality verification requires that information.

This architecture is intended to reduce cross-concern reasoning leakage. If a single evaluator is asked to judge multiple concerns, its free-form explanation can mix unrelated failure reasons. SRG instead isolates responsibilities structurally and passes only the Boolean verdict and concise reason needed by downstream logic.

## 5. Decision Logic: Risk Scores vs. Boolean Contracts

CareGuardAI uses numerical risk assessment and releases a response when  $SRA \leq 2$  and  $HRA \leq 2$ . This provides a graded risk representation and is well suited to its clinical-safety objective. [cite](#) [turn2view0](#)

SRG deliberately avoids a numerical risk score as the final control contract. Each gate returns a Boolean verdict:  $v1$  for policy/tone/completeness and  $v2$  for factual correctness/confidentiality. The application computes  $\text{approve} = v1 \text{ AND } v2$ . This makes the final release rule easy to audit, test, reproduce, and explain. It also prevents the LLM from becoming the final decision authority through an arbitrary numerical score.

This should be presented as a design trade-off rather than an absolute superiority claim. Numerical scoring can express degrees of risk, while SRG's Boolean contract prioritizes deterministic release control and simpler auditability.

## 6. Confidentiality as a Dedicated Architectural Boundary

SRG treats confidentiality differently from a generic safety score. A deterministic pattern-based check can identify recognizable sensitive formats such as account numbers, card-like numbers, emails, and phone numbers. The Redactor creates a sanitized representation before the Review Agent sees the response. The Error Checker can additionally perform a hard confidentiality check on the original response before semantic evaluation.

This produces a two-layer confidentiality strategy: protect the Review Agent from unnecessary exposure to sensitive spans, while independently checking the original output for actual leakage. The separation is especially appropriate for customer-support systems, where generated responses can contain customer-specific identifiers.

### 7. Policy Grounding Difference

CareGuardAI's controller derives safety constraints from the medical context and vulnerability screening. SRG instead uses retrieval-based policy grounding for organizational support rules. Policy documents are embedded using sentence-transformer representations and indexed with FAISS; relevant policy context  $P$  is retrieved and supplied to the semantic verification stage.

This distinction changes the role of context. CareGuardAI uses context primarily to understand patient vulnerability and constrain generation. SRG uses retrieved organizational policy context primarily to verify whether the already-generated support response complies with applicable rules.

### 8. Refinement and Escalation

| Stage                 | CareGuardAI                                       | SRG                                             |
|-----------------------|---------------------------------------------------|-------------------------------------------------|
| Initial failure       | Risk threshold not satisfied                      | v1 or v2 is false                               |
| Feedback              | Risk-evaluator feedback                           | Failing gate reason                             |
| Rewrite               | Iterative refinement                              | Self-Refine                                     |
| Maximum iterations    | Up to 3 iterations described in the decision loop | Maximum 2 attempts                              |
| Re-verification       | Risk evaluation repeated                          | Redaction + both gates rerun from the beginning |
| Final unresolved case | Blocked                                           | Human escalation                                |

Table 2. Refinement and failure-handling comparison.

An important SRG control is that a rewritten response is not trusted merely because it has been modified. The refined response is reintroduced to the complete verification pipeline, including redaction and both gates. This prevents a rewrite from bypassing a newly introduced confidentiality, policy, or factual problem.

### 9. Why SRG Can Be Defended as Better for the Target Use Case

The defensible claim is not that SRG is better than CareGuardAI in every dimension. CareGuardAI is specifically optimized for patient-facing healthcare and includes a sophisticated vulnerability-screening controller. SRG is better aligned with customer-support response verification because its architecture directly models customer-support concerns: organizational policy compliance, factual correctness, confidentiality, response quality, and controlled human escalation.

| Customer-Support Need                    | SRG Mechanism                       | Architectural Benefit                            |
|------------------------------------------|-------------------------------------|--------------------------------------------------|
| Prevent exposure of customer identifiers | Redactor + confidentiality check    | Sensitive information receives explicit handling |
| Keep policy review focused               | Review Agent sees $r' + P$          | Reduced exposure and concern separation          |
| Verify factual support information       | Error Checker sees original $r + P$ | Independent factual verification                 |
| Avoid a single LLM deciding release      | v1 AND v2 in application code       | Deterministic release condition                  |
| Correct rather than immediately reject   | Self-Refine                         | Automatic recovery for correctable failures      |
| Prevent infinite retries                 | Two-attempt bound                   | Predictable execution                            |
| Handle persistent failures               | Human escalation                    | No silent delivery of denied output              |

Table 3. Customer-support requirements mapped to SRG architectural controls.

## 10. What SRG Does NOT Claim

To keep the research claim academically defensible, SRG should not claim that it universally outperforms CareGuardAI. The two systems target different domains and risk models. CareGuardAI has benchmark evidence on medical safety and hallucination datasets, while the current SRG evidence is preliminary and based on a small synthetic evaluation set. A direct performance superiority claim would therefore require a controlled head-to-head experiment.

The appropriate research claim is architectural and use-case specific: SRG proposes a response-verification architecture that adapts inference-time multi-agent guardrails to customer-support settings and adds explicit information isolation, deterministic Boolean approval, dedicated confidentiality controls, retrieval-grounded organizational policy checking, and bounded refinement with escalation.

## 11. Research Gap and Proposed Contribution

CareGuardAI establishes that inference-time multi-agent guardrails can combine specialized evaluation and iterative refinement for safety-critical LLM responses. SRG builds on that direction but changes the architecture around the operational requirements of customer support. The proposed contribution is the combination of response-level policy verification, factual/confidentiality verification, information isolation, deterministic Boolean release control, and bounded Self-Refine escalation.

The resulting research question can be framed as: Can a structurally isolated, response-level two-gate guardrail provide an auditable and practical mechanism for verifying customer-support LLM responses while preventing confidentiality leakage and allowing bounded automatic correction?

## 12. Defense-Ready Explanation

If asked, “Isn't SRG just CareGuardAI with different names?”, the technically correct response is: “Both systems use inference-time multi-agent verification and refinement, so that shared foundation is acknowledged. SRG differs in what each gate is allowed to see and decide. The Review Agent operates on a redacted response for policy, tone, and completeness; the Error Checker independently inspects the original response for factual correctness and confidentiality; and deterministic application logic requires both Boolean gates to pass. SRG therefore changes the information-flow architecture and decision contract rather than merely renaming the evaluators.”

If asked, “Why not use CareGuardAI directly?”, the answer is: “CareGuardAI is designed around clinical safety and hallucination risk in patient-facing healthcare. SRG addresses a different operational environment where confidentiality and organizational policy are central. Its architecture is therefore intentionally optimized for response-level customer-support verification rather than clinical triage.”

## 13. Suggested Architecture Diagram for the Review

CareGuardAI: Query → Controller / Vulnerability Screening → Safety-Constrained Generator → SRA + HRA → Threshold Decision → Refine / Block.

SRG: Query → Support LLM → Candidate Response → Redactor → {Review Agent on  $r'$  + P; Error Checker on  $r$  + P} → approve =  $v1 \text{ AND } v2$  → Deliver / Self-Refine ( $\leq 2$ ) / Human Escalation.

The visual comparison should emphasize that CareGuardAI's main control axis is risk-aware clinical context and numerical thresholds, while SRG's main control axis is information-isolated response verification and deterministic Boolean release.

## 14. Final Positioning

SRG should be positioned as an architectural extension and domain adaptation of inference-time multi-agent guardrail research, not as a replacement that universally dominates CareGuardAI. The strongest novelty argument is the combination of (1) explicit information isolation between verification responsibilities, (2) deterministic Boolean

